LLM/Agent-Guided Equality Saturation and Superoptimization

Research overview — compiled September 2026. Several cited works are 2025–2026 workshop papers or preprints; treat empirical numbers as preliminary until you've read the papers directly.

---

1. Overview

Equality saturation represents a program (or expression, or circuit, or heuristic) as an **e-graph**: a compact structure holding an exponential number of equivalent rewritten forms without committing to any one of them. Rewrite rules are applied until the e-graph *saturates* (no new equivalences can be added, or a budget runs out), and then an **extraction** step picks the lowest-cost representative. The appeal is that it sidesteps the classic phase-ordering problem of greedy rewriting — you never have to guess which rewrite to apply first.

The catch is scale. Two things blow up:

- **Search/exploration**: a rich ruleset applied exhaustively makes the e-graph grow combinatorially, often exhausting memory before saturation.
- **Extraction**: even from a finished e-graph, picking the globally lowest-cost term while respecting shared sub-expressions is NP-hard in general.

Both problems have historically been solved with hand-written scaffolding: manually partitioned rulesets, manually scheduled saturation phases, manually designed cost models, and — in the state of the art just before LLMs entered the picture — manually written "guides" that tell the system which intermediate checkpoint to aim for next (Koehler et al., *Guided Equality Saturation*, POPL 2024).

The research direction you're asking about replaces that hand engineering with a language model or agent, while keeping the e-graph (or some other formally checkable substrate) as the thing that actually guarantees soundness. Two threads have emerged in 2025–2026:

1. **LLM-guided equality saturation proper** — the e-graph stays the core data structure; an LLM is inserted as an intermediate reasoning layer that proposes high-level checkpoints, and the e-graph fills in the low-level, provably sound rewrite chain between them. This is exactly the pattern in the 2025 paper you described — it's a real, specific paper (see §4.4).
2. **Agentic code optimization / compiler-LLM cooperation** — a broader framing where the "formal substrate" isn't necessarily an e-graph; it can be any set of compiler passes exposed as callable tools, orchestrated by one or more LLM agents, with correctness delegated to generated tests or to the underlying passes themselves rather than to e-graph equivalence proofs.

Both threads share the same underlying bet: LLMs are good at *semantic, high-level* reasoning about "what should this program become" but bad at guaranteeing *low-level correctness*; formal rewrite systems are the reverse. Put them in the same loop and each covers the other's weakness.

---

2. Why This Is a Genuinely New Subfield

It's worth being precise about why this isn't just a rebrand of "ML for compilers," because the term has been used loosely for a decade.

Generation	Representative work	What the model/agent actually decides	What guarantees correctness
Classical ML-for-compilers	RL-based phase ordering (AutoPhase-style), learned cost models, Halide/TVM auto-scheduling	A scalar or low-dimensional knob: pass order, schedule parameter, cost estimate	Nothing new — the model only ever chooses among moves a human already proved sound; it's statistics wrapped around a fixed, safe move set
LLM-as-optimizer	Cummins et al., *Large Language Models for Compiler Optimization* (2023); direct "LLM rewrites this function" pipelines	The entire output program or optimization sequence, in one shot	Nothing built in — correctness is checked post hoc by tests, if at all, and incorrect output is a known failure mode
LLM/agent + formal rewrite substrate	LGuess, ASPEN, LLM-Guided Strategy Synthesis, Agentic Code Optimization, Magellan	High-level *structure*: which checkpoint, which strategy/schedule, which heuristic template to aim for	The e-graph (or a compiler pass, or an autotuner) fills in and checks the low-level steps, so soundness never depends on the LLM being right

The third row is the new thing. AwareCompiler (2025) frames this explicitly as a shift "from data-specific learning to generalizable knowledge-based reasoning" — the model is no longer being trained to predict a number, it's being used as a reasoning engine whose outputs are architecturally prevented from corrupting correctness. That architectural separation — not the mere presence of an LLM — is what makes this a distinct research program with its own open problems (see §5), rather than a bigger model bolted onto an old pipeline.

---

3. Research Plan

A phased plan, assuming you want to go from "read the literature" to "have a concrete prototype and a paper-shaped contribution."

Phase 0 — Environment and grounding (1–2 weeks)

- Stand up `egg` (Rust) or `egglog` and work through their tutorials on a small domain (arithmetic simplification is enough to build intuition).
- Read, in full: Tate et al. 2009 (original equality saturation), Willsey et al. 2021 (egg), Koehler et al. 2024 (*Guided Equality Saturation*), and the two papers you already identified — LGuess (arXiv 2511.00403) and Mikek et al.'s *Agentic Code Optimization via Compiler-LLM Cooperation* (arXiv 2604.04238).
- Pick one narrow target domain to avoid scope creep. Two reasonable options given your existing LLVM IR tooling from the leakage-surface project: (a) a small LLVM peephole/instcombine-style rewrite domain, reusing IR you already parse; or (b) a self-contained symbolic domain like boolean/bitvector expressions, which is easier to get end-to-end quickly.

Phase 1 — Reproduce a minimal baseline (2–3 weeks)

- Build a stripped-down version of the LGuess loop: e-graph + LLM checkpoint proposal + saturation between checkpoints, on your chosen toy grammar.
- Implement two baselines to compare against: plain equality saturation with a fixed schedule, and "ask the LLM directly for the whole rewrite chain, no e-graph." Both are what LGuess itself compares to, so you get free calibration against a real published result.
- Goal of this phase is intuition, not novelty: see where checkpoints get hallucinated-but-unreachable, where the e-graph still blows up despite guidance, and how sensitive results are to checkpoint granularity.

Phase 2 — Pick and formalize a gap (2–4 weeks)

- Choose one item from §5 below. Write a one-page problem statement, hypothesis, and evaluation plan *before* building further — this is the point where a vague direction becomes a falsifiable research question.
- Two candidates worth flagging specifically: (a) the sample-efficiency gap in checkpoint extraction (§5.2), which is a clean, scoped systems problem; and (b) the constant-time/security angle (§5.7), which is more ambitious but connects directly to tooling you already have running (TIMECOP, Binsec/Rel2, your GNN leakage predictor).

Phase 3 — Prototype (4–6 weeks)

- Build the minimal system needed to test the Phase 2 hypothesis. Resist the urge to generalize beyond what the evaluation plan requires.

Phase 4 — Evaluate and write up (2–3 weeks)

- Compare against the Phase 1 baselines, not just against "no guidance at all" — that comparison is where most of the papers in §4.4 are currently thin, so doing it carefully is itself a contribution.

Phase 5 — Venue

- The EGRAPHS workshop (co-located with PLDI) is the natural home for anything squarely about equality saturation — it's a small, receptive community that has already hosted both LGuess and the ML-guided-equality-saturation talk. If the constant-time angle pans out, CGO or COSADE workshops (which you're already targeting for LeakSurface) are a better fit for the security framing, with EGRAPHS as a secondary option.

---

4. Current Literature

4.1 Foundations: e-graphs and equality saturation

- **Tate, Stepp, Tatlock, Lam. "Equality Saturation: A New Approach to Optimization."** POPL 2009. Introduced the technique and the ILP formulation of extraction.
- **Willsey, Nandi, Wang, Flatt, Tatlock, Panchekha. "egg: Fast and Extensible Equality Saturation."** POPL 2021. The modern, language-agnostic e-graph library (deferred congruence closure via "rebuilding") that reignited the field; most subsequent work, including everything in §4.4, is built on or benchmarked against it.
- **Zhang, Wang, Flatt, Cao, Zucker, Rosenthal, Tatlock, Willsey. "Better Together: Unifying Datalog and Equality Saturation" (egglog).** PLDI 2023. Merges equality saturation with Datalog-style relational e-matching for scalability; a plausible substrate for agent-driven search since goals can be expressed as relational queries over the e-graph rather than tree patterns.
- **Yang, Phothilimthana, Wang, Willsey, Roy, Pienaar. "Equality Saturation for Tensor Graph Superoptimization" (TENSAT).** MLSys 2021 (arXiv 2101.01332). Canonical e-graph superoptimization result; showed that greedy extraction can pick a *worse*-than-original graph because it ignores shared sub-expressions — the concrete motivation for ILP extraction that nearly every later paper cites.

4.2 Human-guided equality saturation (direct precursor to LLM "checkpoints")

- **Koehler, Trinder, Steuwer. "Sketch-Guided Equality Saturation."** 2022 (arXiv 2111.13040). A human supplies intermediate "sketches" the search must pass through to control direction on complex functional-program optimizations.

- **Koehler, Goens, Bhat, Grosser, Trinder, Steuwer. "Guided Equality Saturation."** POPL 2024 (selected for MIT PL Review 2024). Formalizes the general "guide" abstraction: break a hard rewrite problem into a sequence of saturations connected by human-specified checkpoints. This is precisely the role that LGuess and its successors have an LLM step into.

4.3 ML/search-guided extraction and construction (pre-LLM)

- **Extraction complexity**: extraction that accounts for shared sub-expressions is NP-hard in general (formalized via the ILP encoding in Tate et al.; complexity results further characterized by Goharshady et al. 2024, cited across several 2025 papers).
- **Cai et al. "SmoothE: Differentiable E-Graph Extraction."** ASPLOS 2025. Relaxes discrete extraction into a differentiable objective so learned cost models can be optimized directly rather than plugged into a fixed ILP/greedy extractor.
- **Yin, Song, Chen, Cai, Zhang, Yu. "e-boost: Boosted E-Graph Extraction with Adaptive Heuristics and Exact Solving."** 2025 (arXiv 2508.13020). Hybrid adaptive-heuristic-plus-exact-solver extraction, a pragmatic answer to NP-hardness.
- **"Optimizing Tensor Computation Graphs with Equality Saturation and Monte Carlo Tree Search."** 2024 (arXiv 2410.05534). Uses MCTS — not an LLM — to decide which rewrites to apply; a direct statistical (non-LLM) predecessor to the checkpoint/strategy-selection idea.
- **Nandi, Willsey, Zhu, Wang, Saiki, Anderson, Schulz, Grossman, Tatlock. "Rewrite Rule Inference Using Equality Saturation" (Ruler).** OOPSLA 2021, Distinguished Paper Award. Uses equality saturation to *synthesize the rewrite rules themselves*, with SMT-checked soundness — complementary to LLM-guided *search over* a fixed ruleset (see §5.3).
- **Pal, Saiki, Tjoa, Richey, Zhu, Flatt, Willsey, Tatlock, Nandi. "Equality Saturation Theory Exploration à la Carte."** OOPSLA 2023. Extends Ruler-style rule/lemma discovery.

4.4 Core cluster: LLM-guided equality saturation (2025–2026)

- **Ji et al. "Equality Saturation Guided by Large Language Models" (LGuess).** EGRAPHS 2025 workshop / PLDI 2025 (arXiv 2511.00403). This is the paper matching your description exactly. E-graphs sit as an intermediate layer between an LLM and the rewrite system; the LLM is queried only for high-level checkpoints (generalizing Koehler et al.'s human-authored guides), and the e-graph supplies the low-level, sound rewrite chain between them. The core technical difficulty is that checkpoints must be *extracted* from a saturated e-graph, and raw LLM output isn't directly usable as one — LGuess addresses this by learning a probabilistic model from LLM outputs to make extraction tractable. Evaluated on multivariable polynomial factorization; beats both plain equality saturation and "ask the LLM for the full chain directly."
- **"Machine Learning Guided Equality Saturation."** EGRAPHS 2025 workshop talk. A closely related, more general framing: learn Koehler et al.'s "guides" via ML (not necessarily an LLM specifically) to reduce expert dependence.
- **Zhang, Deng, Kuehn, Ho, Yu, Zhang, Ren. "ASPEN: LLM-Guided E-Graph Rewriting for RTL Datapath Optimization."** MLCAD 2025. Same LLM-proposes/e-graph-verifies pattern, applied to RTL datapath optimization, replacing both hand-designed rewrite rules and proxy power-performance-area cost models.
- **Yin, Xiao, Luo, Zou, Liang. "LLM-Guided Strategy Synthesis for Scalable Equality Saturation."** 2026 (arXiv 2604.17364). Broadens the target from individual checkpoints to full *strategies* — rule partitioning, saturation scheduling, e-graph state simplification, and search-budget allocation — synthesized by an LLM rather than hand-designed, aimed at general e-graph-based compilers rather than one domain.

4.5 Broader cluster: agentic code optimization / compiler-LLM cooperation (2025–2026)

- **Mikek, Vashchilenko, Lu, Xu (AWS AI / Georgia Tech). "Agentic Code Optimization via Compiler-LLM Cooperation."** 2026 (arXiv 2604.04238). This is the paper behind the "genuinely new subfield" framing you mentioned. A multi-agent system: one LLM optimization agent per abstraction level, individual compiler passes exposed as callable tools, an LLM test-generation agent that probes correctness and performance, and an orchestrating "guiding" LLM that allocates compute budget across levels. Reports up to 1.25× speedup over both standard compiler optimization and level-specific LLM baselines. It generalizes LGuess's idea from "e-graph as the one formal substrate" to "any compiler pass/tool the agent can call," at the cost of trading e-graph-verified equivalence for test-suite-verified equivalence.
- **Chen et al. "Magellan: Autonomous Discovery of Novel Compiler Optimization Heuristics with AlphaEvolve."** 2026 (arXiv 2601.21096). Different mechanism — evolutionary program search via AlphaEvolve plus an LLM coding agent plus an external autotuner for numeric hyperparameters — but the same structural idea: the LLM proposes high-level heuristic *logic*, and a separate, more mechanical process (autotuning) fills in the low-level numeric detail. Targets LLVM function-inlining and register-allocation heuristics; matches or beats decades-old hand-tuned baselines.
- **"AwareCompiler: Agentic Context-Aware Compiler Optimization via a Synergistic Knowledge-Data Driven Framework."** 2025 (arXiv 2510.11759). Explicitly frames the shift from data-specific learned models to generalizable knowledge-based agent reasoning as the field's core distinguishing move.
- Worth tracking but more tangential: **Astra** (multi-agent GPU kernel optimization, arXiv 2509.07506), **VecTrans** (LLM-assisted auto-vectorization, arXiv 2503.19449) and **LLM-Vectorizer** (verified loop vectorizer, CGO 2025) as examples of "LLM proposes, a checker verifies" without e-graphs, and a 2026 evaluation study on whether coding agents can rediscover known-missed LLVM peephole optimizations unaided (arXiv 2607.02684) — a useful empirical floor for "how good is an LLM alone" before adding any scaffolding.

4.6 Superoptimization lineage (context)

- **Massalin (1987)** — the original brute-force peephole superoptimizer.
- **Joshi, Nelson, Randall. "Denali: A Goal-Directed Superoptimizer."** PLDI 2002. Used automatic theorem proving and an e-graph-like structure for goal-directed search; the direct conceptual ancestor cited by nearly every modern e-graph extraction paper.
- **Schkufza, Aiken, Sharma. "Stochastic Superoptimization" (STOKE).** ASPLOS 2013. MCMC search over instruction sequences — a useful contrast case (stochastic search vs. equality saturation's exhaustive-but-compact representation).
- **Kuepper, Erbsen, Gross, Conoly, Sun, Tian, Wu, Chlipala, Chuengsatiansup, Genkin, Wagner, Yarom. "CryptOpt: Verified Compilation with Randomized Program Search for Cryptographic Primitives."** PLDI 2023. Superoptimization specifically for cryptographic code, with formal verification — directly relevant if you pursue the constant-time angle in §5.7.

---

5. Work to Be Done / Open Problems

5.1 Generalization beyond toy domains. LGuess is evaluated on polynomial factorization; the strategy-synthesis and RTL papers each pick one domain too. Nobody has yet shown checkpoint-based LLM guidance working on a large, messy, real compiler IR (LLVM/MLIR) with control flow, memory, and side effects — the domains where equality saturation has historically struggled most.

5.2 Checkpoint-extraction sample efficiency. LGuess's probabilistic model exists specifically because raw LLM checkpoint proposals are too expensive to use directly, requiring many LLM calls per saturation round. Distilling this into a smaller, amortized model, or having the LLM read a compressed e-graph summary instead of needing repeated extraction-then-query loops, is an open and fairly tractable systems problem.

5.3 Rule-level soundness, not just chain-level soundness. E-graph-guaranteed correctness only covers what's expressible within the *given* ruleset — it says nothing about whether each rule in that ruleset is itself sound (that's Ruler's and CCLemma's job, via SMT/inductive proof). Nobody has yet combined LLM-guided *search* over a ruleset with LLM- or agent-proposed *new* candidate rules that are then independently SMT-verified before being trusted.

5.4 Unifying checkpoints and strategies. LGuess-style checkpoint proposals and the "Strategy Synthesis" paper's rule-partitioning/scheduling are two different levers on the same explosion problem, studied separately. An agent that jointly decides both — which checkpoint to aim for *and* which rule subset/schedule to use to get there — is an open combination.

5.5 Agent participation in the cost model itself. Current work uses fixed size, PPA-proxy, or SmoothE-style learned cost models for extraction. Whether an LLM/agent should also help select or adapt the cost model (a much more "creative," context-dependent judgment than emitting a rewrite) is unexplored.

5.6 Bridging the two verification philosophies. The equality-saturation cluster (§4.4) trusts e-graph equivalence proofs; the broader agentic cluster (§4.5) trusts LLM-generated test suites, which have no formal guarantee. A system that uses e-graph verification for the sub-parts of a pipeline that go through a term-rewriting IR, and falls back to agentic test-generation only where no formal substrate exists, would be a genuine synthesis of the two threads that the current literature keeps largely separate.

5.7 A security/constant-time niche. None of the papers above use a *security property* as the thing being preserved — correctness in this literature always means "same output" or "same performance-proxy." Given your existing LeakSurface tooling (GNN leakage predictor, TIMECOP, Binsec/Rel2, ct-fuzz, LLVM IR pipeline), a plausible and currently unoccupied direction is an LLM-guided equality-saturation system where checkpoints must be constant-time-preserving, and the e-graph only explores rewrite chains inside a verified constant-time-equivalence class — using your existing side-channel tooling as the correctness oracle in place of (or alongside) unit tests. This would connect naturally to CryptOpt's verified-search precedent (§4.6) without duplicating it, since CryptOpt is stochastic search, not equality saturation. Flagging this as one option among several, not a foregone direction — it's the one place in this space that overlaps with work you're already doing, which cuts setup cost, but it's also the most ambitious item on this list.

5.8 Shared benchmarks. There's no equivalent of SPEC/MLPerf for "LLM-guided equality saturation" yet — every paper picks its own domain and baseline. A comparative benchmark suite would be a modest but genuinely useful contribution on its own.

5.9 Ablating "LLM" against "any external guide." LGuess compares against plain equality saturation and "LLM writes the whole chain," but not against non-LLM ML guides (SmoothE, MCTS-based rewrite selection) or against Koehler et al.'s original human-authored guides. It's not yet established how much of the benefit comes from semantic/LLM knowledge specifically versus simply having *some* external guide at all — an important and currently missing ablation.

---

6. References (with links)

- Tate, Stepp, Tatlock, Lam. *Equality Saturation: A New Approach to Optimization.* POPL 2009.

- Willsey, Nandi, Wang, Flatt, Tatlock, Panchekha. *egg: Fast and Extensible Equality Saturation.* POPL 2021. https://doi.org/10.1145/3434304
- Zhang, Wang, Flatt, Cao, Zucker, Rosenthal, Tatlock, Willsey. *Better Together: Unifying Datalog and Equality Saturation.* PLDI 2023. https://arxiv.org/abs/2304.04332
- Yang, Phothilimthana, Wang, Willsey, Roy, Pienaar. *Equality Saturation for Tensor Graph Superoptimization.* MLSys 2021. https://arxiv.org/abs/2101.01332
- Koehler, Trinder, Steuwer. *Sketch-Guided Equality Saturation.* 2022. https://arxiv.org/abs/2111.13040
- Koehler, Goens, Bhat, Grosser, Trinder, Steuwer. *Guided Equality Saturation.* POPL 2024. https://doi.org/10.1145/3632900
- Cai et al. *SmoothE: Differentiable E-Graph Extraction.* ASPLOS 2025.
- Yin, Song, Chen, Cai, Zhang, Yu. *e-boost: Boosted E-Graph Extraction with Adaptive Heuristics and Exact Solving.* 2025. https://arxiv.org/abs/2508.13020
- *Optimizing Tensor Computation Graphs with Equality Saturation and Monte Carlo Tree Search.* 2024. https://arxiv.org/abs/2410.05534
- Nandi, Willsey, Zhu, Wang, Saiki, Anderson, Schulz, Grossman, Tatlock. *Rewrite Rule Inference Using Equality Saturation.* OOPSLA 2021. https://arxiv.org/abs/2108.10436
- Pal, Saiki, Tjoa, Richey, Zhu, Flatt, Willsey, Tatlock, Nandi. *Equality Saturation Theory Exploration à la Carte.* OOPSLA 2023.
- Ji et al. *Equality Saturation Guided by Large Language Models.* EGRAPHS/PLDI 2025. https://arxiv.org/abs/2511.00403
- Zhang, Deng, Kuehn, Ho, Yu, Zhang, Ren. *ASPEN: LLM-Guided E-Graph Rewriting for RTL Datapath Optimization.* MLCAD 2025.
- Yin, Xiao, Luo, Zou, Liang. *LLM-Guided Strategy Synthesis for Scalable Equality Saturation.* 2026. https://arxiv.org/abs/2604.17364
- Mikek, Vashchilenko, Lu, Xu. *Agentic Code Optimization via Compiler-LLM Cooperation.* 2026. https://arxiv.org/abs/2604.04238
- Chen et al. *Magellan: Autonomous Discovery of Novel Compiler Optimization Heuristics with AlphaEvolve.* 2026. https://arxiv.org/abs/2601.21096
- *AwareCompiler: Agentic Context-Aware Compiler Optimization via a Synergistic Knowledge-Data Driven Framework.* 2025. https://arxiv.org/abs/2510.11759
- Joshi, Nelson, Randall. *Denali: A Goal-Directed Superoptimizer.* PLDI 2002.
- Schkufza, Aiken, Sharma. *Stochastic Superoptimization.* ASPLOS 2013.
- Kuepper et al. *CryptOpt: Verified Compilation with Randomized Program Search for Cryptographic Primitives.* PLDI 2023.